

Algorithms for Data Science

Exercise 4

See the cheatsheet for valuable remarks!

Task 4.1: Dynamic Programming: Nontouching Cells (1D)

You are given an array consisting of n non-negative values. Your task is to choose a subset of the array cells such that no two cells touch and the total value is maximized. It suffices that you output the total value.

2	1	1	2	0
---	---	---	---	---

An example array ($n = 5$). The chosen cells are shaded in gray. The total value is 4.

Input: array $val[1..n]$.

Output: total value of the chosen array cells.

Solve the problem via the following methods. What is the running time and memory complexity?

- (a) Backtracking
- (b) Recursion with Memoization
- (c) Dynamic Programming
- (d) Dynamic Programming (with only $O(1)$ extra space!)
- (e) Modify the DP from subtask (c) such that it outputs the chosen grid cells (**Output:** binary array $chosen[1..n]$ where $chosen[i] = true$ if the grid cell in the i -th column has been chosen.)

Task 4.2: Dynamic Programming: Nontouching Cells (2D)

Consider the following generalization of the last task (Task 4.1). You are given a $(k \times n)$ -grid consisting of n columns and k rows. Each grid cell has some non-negative value. Your task is to choose **at most one** grid cell from each of the n columns such that no two grid cells are touching each other (touching with corners is also forbidden) and the total value is maximized. It suffices that you output the total value.

0	1	2	1	0
1	1	1	1	0
0	1	2	1	2
0	1	2	1	2

An example grid ($n = 5$ and $k = 4$). The chosen cells are shaded in gray. The total value is 6.

Input: array $val[1..n][1..k]$.

Output: total value of the chosen grid cells.

Solve the problem for any $k \geq 4$ via the following methods. What is the running time?

- (a) Backtracking
Hint: First solve the problem for the restricted case where the input specifies some $j \in \{1, \dots, k\}$ and, within the *last* column, we are allowed to pick only the grid cell of the j -th row.

- (b) Recursion with Memoization
- (c) Dynamic Programming
- (d) Dynamic Programming with $O(k)$ extra memory
- (e) (Bonus Task) Modify the algorithm of task (c) such that it outputs the chosen grid cells as a binary array $chosen[1..n][1..k]$ where $chosen[i][k] = true$ if the grid cell in the i -th column and j -th row has been chosen.

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 4.3: Dynamic Programming: Maximum Block

Given an array containing possibly negative values, compute the value of a block of maximum total value.

Formally, let $a[1..n]$ be the input array. The problem is to output the total value of a *heaviest* block, that is, of a subarray $a[i..j]$ of maximum total value $\sum_{k=i}^j a[k]$.

Task 4.4: Heap Sort

Run heap sort on the array $a = \langle 20, 10, 30, 5, 40 \rangle$. Draw the array a after each single change as a binary complete tree.

Task 4.5: Quick Sort: Polish Flag

Second May is Polish National Flag Day. For this occasion, you want to program a robot to arrange red and white balls to form a Polish flag. In particular, you are given $2n$ bins numbered from 1 to $2n$ where each bin contains exactly one ball. In total there are n red and n white balls. Your goal is that the first n bins contain red balls, the remaining ones white balls. Your robot can only do the following operations:

- $\text{isRed}(i)$: the robot looks at the i -th bin and returns true if it contains a red ball, and false otherwise.
- $\text{swap}(i, j)$: the robot exchanges the balls of the i -th and j -th bin.

Furthermore, your robot has only a small memory of constant size.

- (a) How is this problem related to quick sort?
- (b) Program the robot to arrange a Polish flag within $O(n)$ operations.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 4.6: Fibonacci Numbers

We define the Fibonacci numbers as follows:

$$F_n = \begin{cases} n & n = 0, 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases}$$

- (a) How many times do we perform an addition if compute F_n recursively with the formula above?
- (b) Give an algorithm that computes F_n using only $O(\log n)$ arithmetic operations. Use the following recurrence:

$$\text{For } n > 1, \quad F_{2n-1} = F_n^2 + F_{n-1}^2 \quad \text{and} \quad F_{2n} = F_n^2 + 2F_n F_{n-1} .$$

- (c) Give an algorithm that computes F_n using only $O(\log n)$ arithmetic operations. Use the following matrix recurrence:

$$\text{For } n > 1, \quad \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix} .$$

Note: In the RAM model, each basic arithmetic operation costs $O(1)$ time. This is realistic as long as the values are not too large. However, in the Fibonacci algorithms above such an assumption is not realistic as the n -th Fibonacci number has $\Theta(n)$ bits in its standard encoding, and consequently it takes $\Theta(n)$ time just to write it down, not to mention the time to compute it. Thus, to be more realistic, we should assume that the addition of two n -bit numbers takes $\Theta(n)$ time, whereas multiplication takes $O(n \log n)$ time. (It is an open question in computer science whether one can do the multiplication faster. The best-known $O(n \log n)$ algorithm for multiplication is relatively young from 2019.) Summarized, the best-known algorithm to compute the n -Fibonacci number takes $O(n \log^2 n)$ time.